TP3 : Blockchain, cryptographie et intégrité des données

24 octobre 2026

1 Objectifs du TP

Ce TP a pour objectif de concevoir et d'expérimenter une blockchain simplifiée en Python afin de comprendre les mécanismes cryptographiques qui contribuent à la sécurité et à l'intégrité des données.

À la fin de ce TP, l'étudiant devra être capable de :

- créer des blocs liés par des empreintes cryptographiques ;
- détecter la modification d'un bloc ou d'une transaction ;
- construire un arbre de Merkle et vérifier une transaction ;
- implémenter un mécanisme simplifié de preuve de travail ;
- comprendre le rôle des signatures numériques ;
- analyser les limites de sécurité d'une blockchain simplifiée.

2 Prérequis

- Python 3.8 ou supérieur ;
- notions de programmation orientée objet ;
- notions de fonctions de hachage et de cryptographie.

3 Installation

Créez un fichier `requirements.txt` :

```
pycryptodome
```

Installez les dépendances :

```
python -m pip install -r requirements.txt
```

4 Structure du projet

Organisez votre projet comme suit :

```
my_blockchain_project/
|-- node/
| |-- __init__.py
| |-- utils.py
| |-- block.py
| |-- merkle.py
|-- main.py
|-- requirements.txt
```

5 Partie I : Création d'une blockchain

5.1 Étape 1 : Fonction de hachage

Dans `node/utils.py`, ajoutez :

```
from Crypto.Hash import SHA256

def calculate_hash(data: bytes) -> str:
    h = SHA256.new(data)
    return h.hexdigest()
```

La fonction SHA-256 transforme des données en une empreinte de taille fixe. Une modification des données produit généralement une empreinte différente.

5.2 Étape 2 : Implémentation d'un bloc

Dans `node/block.py`, ajoutez :

```
import json
from node.utils import calculate_hash

class Block:
    def __init__(self, timestamp, transactions,
                 previous_hash="", nonce=0):
        self.timestamp = timestamp
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        content = {
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce
        }
        data = json.dumps(
            content, sort_keys=True
        ).encode("utf-8")
        return calculate_hash(data)
```

Le champ `previous_hash` référence l'empreinte du bloc précédent. Le champ `nonce` sera utilisé dans la preuve de travail.

5.3 Étape 3 : Création de plusieurs blocs

Dans `main.py`, ajoutez :

```
import time
from node.block import Block

block1 = Block(
    time.time(),
    ["Transaction A", "Transaction B"]
)

block2 = Block(
    time.time(),
    ["Transaction C"],
    block1.hash
)
```

```
block3 = Block(
    time.time(),
    ["Transaction D"],
    block2.hash
)

for block in [block1, block2, block3]:
    print("Transactions :", block.transactions)
    print("Previous hash :", block.previous_hash)
    print("Block hash :", block.hash)
    print("-" * 50)
```

Travail demandé :

1. Exécutez le programme.
2. Créez deux blocs supplémentaires.
3. Expliquez le rôle de l'empreinte du bloc précédent.
4. Modifiez une transaction après la création d'un bloc.
5. Comparez la nouvelle empreinte avec l'empreinte initiale.

6 Partie II : Détection d'anomalies

Un attaquant peut essayer de modifier une transaction déjà enregistrée. Le hachage permet de détecter cette modification, mais une blockchain complète doit également vérifier la cohérence des liens entre les blocs.

Ajoutez à `main.py` :

```
original_hash = block2.hash

# Simulation d'une modification malveillante
block2.transactions[0] = "Transaction falsifiee"

new_hash = block2.calculate_hash()

print("Empreinte initiale :", original_hash)
print("Nouvelle empreinte :", new_hash)
print("Modification detectee :", original_hash != new_hash)
```

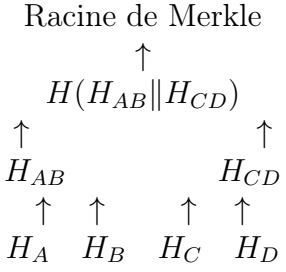
Travail demandé :

1. Vérifiez que la modification change l'empreinte.
2. Expliquez pourquoi le hachage seul ne suffit pas à empêcher un attaquant de recalculer les empreintes.
3. Proposez un algorithme qui vérifie tous les liens entre les blocs.
4. Expliquez la différence entre détecter une modification et empêcher une modification.

7 Partie III : Arbre de Merkle

7.1 Principe

Un arbre de Merkle organise les empreintes des transactions dans une structure arborescente. Les empreintes sont combinées deux par deux jusqu'à obtenir une empreinte unique appelée *racine de Merkle*.



Ici, H représente SHA-256 et $\|$ la concaténation des empreintes.

7.2 Implémentation

Créez le fichier `node/merkle.py` :

```python
from node.utils import calculate_hash

def merkle_root(transactions):
    if not transactions:
        return calculate_hash(b"")

    level = [
        calculate_hash(tx.encode("utf-8"))
        for tx in transactions
    ]

    while len(level) > 1:
        if len(level) % 2 == 1:
            level.append(level[-1])

        next_level = []

        for i in range(0, len(level), 2):
            combined = (level[i] + level[i + 1]).encode()
            next_level.append(calculate_hash(combined))

        level = next_level

    return level[0]
```

7.3 Expérimentation

Dans `main.py`, ajoutez :

```python
from node.merkle import merkle_root

transactions = [
    "Alice -> Bob : 10",
    "Bob -> Charlie : 5",
    "Charlie -> David : 2",
    "David -> Alice : 1"
]

root1 = merkle_root(transactions)
print("Racine de Merkle :", root1)

transactions[1] = "Bob -> Charlie : 50"
root2 = merkle_root(transactions)
print("Nouvelle racine :", root2)

print("Modification detectee :", root1 != root2)
```

Travail demandé :

1. Testez l'arbre avec 2, 3, 4 et 5 transactions.
2. Observez le résultat lorsque le nombre de transactions est impair.
3. Modifiez une seule transaction et comparez les racines.
4. Expliquez pourquoi la racine de Merkle est utile pour vérifier l'intégrité d'un grand ensemble de transactions.
5. Recherchez le principe d'une preuve de Merkle : comment vérifier l'appartenance d'une transaction sans transmettre toutes les transactions ?

8 Partie IV : Preuve de travail (Proof of Work)

8.1 Principe

La preuve de travail consiste à rechercher une valeur `nonce` qui produit une empreinte satisfaisant une condition donnée.

Dans ce TP, on recherche une empreinte commençant par un certain nombre de zéros. Cette activité illustre le coût de calcul associé au minage, sans reproduire la complexité d'un réseau réel.

8.2 Implémentation

Ajoutez cette méthode à la classe `Block` dans `node/block.py` :

```
    def mine_block(self, difficulty=3):
        target = "0" * difficulty

        while not self.hash.startswith(target):
            self.nonce += 1
            self.hash = self.calculate_hash()

        print("Bloc mine :", self.hash)
        print("Nonce :", self.nonce)
```

Attention : cette méthode doit être indentée dans la classe `Block`.

Dans `main.py`, testez :

```
block1.mine_block(difficulty=3)
```

Travail demandé :

1. Exécutez le minage avec une difficulté de 2, 3 puis 4.
2. Mesurez le temps d'exécution avec le module `time`.
3. Comparez le nombre d'essais nécessaires.
4. Expliquez pourquoi une difficulté plus élevée augmente généralement le travail de calcul.
5. Modifiez une transaction après le minage et vérifiez si la preuve de travail reste valide.

9 Partie V : Signatures numériques

9.1 Principe

Une signature numérique permet de vérifier qu'une transaction a été signée par le détenteur d'une clé privée et qu'elle n'a pas été modifiée depuis sa signature.

Elle repose sur une paire de clés :

— une clé privée, conservée secrète ;

— une clé publique, utilisée pour vérifier la signature.

9.2 Exercice cryptographique

À l'aide de la bibliothèque `pycryptodome`, étudiez les modules :
`Crypto.PublicKey.ECC` et `Crypto.Signature.DSS`.

Travail demandé :

1. Générez une paire de clés ECC.
2. Signez une transaction avec la clé privée.
3. Vérifiez la signature avec la clé publique.
4. Modifiez la transaction après sa signature et recommencez la vérification.

5. Expliquez pourquoi la clé privée ne doit jamais être publiée sur la blockchain.

Les clés privées doivent rester confidentielles. Utilisez des données fictives et ne présentez pas cette implémentation pédagogique comme un système de sécurité prêt pour la production.

10 Questions de synthèse

1. Quelle est la différence entre hachage et chiffrement ?
2. Pourquoi le chaînage des blocs permet-il de détecter certaines modifications ?
3. Quel est l'intérêt d'un arbre de Merkle ?
4. Quel est le rôle de la preuve de travail ?
5. Quelle différence existe-t-il entre intégrité, authentification et confidentialité ?
6. Pourquoi une blockchain ne garantit-elle pas à elle seule la confidentialité des données ?
7. Quelles sont les limites d'une blockchain simplifiée sans réseau distribué ni mécanisme de consensus réel ?

11 Livrables

Chaque binôme doit fournir :

— le code source Python ;

— les résultats des expériences ;

— des captures d'écran de l'exécution ;

— une courte analyse des attaques simulées ;

— une conclusion sur les garanties et les limites de sécurité du système.

12 Conclusion

Ce TP permet d'étudier plusieurs mécanismes fondamentaux de la sécurité des blockchains : hachage cryptographique, chaînage des blocs, arbres de Merkle, preuve de travail et signatures numériques.

Il souligne également qu'une empreinte cryptographique ne suffit pas à garantir la sécurité globale d'un système. La validation des blocs, la gestion des clés, le consensus et la protection des données doivent être étudiés conjointement.